International Journal of Data Science and Analytics (2025) 20:4449–4471
https://doi.org/10.1007/s41060-025-00728-9

REGULAR PAPER

Web-based phishing URL detection model using deep learning optimization techniques

Kousik Barik[1] · Sanjay Misra[2] · Raghini Mohan[3]

Received: 21 June 2024 / Accepted: 24 January 2025 / Published online: 8 February 2025
© The Author(s) 2025

Abstract
Phishing is a cyber-attack in which the attacker redirects Internet users to fraudulent websites. Fake websites look very similar to legitimate ones, leading users to trust them and disclose sensitive information. Despite the available methods, these attacks have grown exponentially, emphasizing the need for advanced techniques. This study proposes an EGSO-CNN model to detect web phishing by integrating features and optimizing deep learning (DL) techniques. A novel dataset has been created to address the availability of existing updated phishing datasets. The StandardScaler and Variational Autoencoders (VAE) are employed for preprocessing and feature extraction. The Enhanced Grid Search Optimization (EGSO) technique optimizes the model's performance. The proposed model yields an accuracy of 99.44%, a recall of 99.21%, and an f1-score of 99.32% with low false positive and error rates. The presented model can assist management by selecting effective phishing detection strategies to enhance customer delight.

Keywords Phishing attacks · Deep learning · Optimization techniques · User awareness

1 Introduction

The advancement of Internet services has significantly transformed individuals' daily lives. During the global pandemic, many traditional retail sectors shifted from in-person to online services. Internet service users store sensitive data on their devices or within online systems [1]. These data often contain username, account name, password, and credit card details. Most Organizations digitize their day-to-day operations to facilitate convenient access to diverse online services. The expansion of the Internet and associated services has introduced the issue of data security and the increasing prevalence of sophisticated cyberattacks. Cybercriminals unlawfully acquire users' sensitive information and manipulate them into participating in illegal online actions [2].

Phishing is a type of cyberattack that employs social engineering techniques to exploit users' information by tricking them into clicking on fraudulent websites. This enables cybercriminals to intercept sensitive information and acquire funds without authorization [3]. Phishing remains a prominent and effective cyber-attack, exhibiting consistent growth over the years [4]. Phishing websites are often designed to simulate original URLs by incorporating key elements such as textual descriptions, logos, and company names. Phishing attacks have substantial economic, political, and social implications for individuals, organizations, and governments. Some of these implications include (a) financial loss [5], (b) confidential and proprietary data theft [6], and (c) facilitating other cyberattacks [7]. According to data published by APWG [8], the number of unique web phishing attacks detected in Q3,2024 was 932,923, up from 877,536 in Q2,2024, as depicted in Fig. 1. These data signify an rising trend of phishing attacks.

✉ Sanjay Misra
sanjay.misra@ife.no

Kousik Barik
kousikbarik@gmail.com

Raghini Mohan
mrm@vcet.ac.in

1 Department of Computer Science, Liverpool John Moores University, Liverpool, UK

2 Department of Applied Data Science, Institute for Energy Technology, Halden, Norway

3 Department of Artificial Intelligence and Data Science, Velammal College of Engineering and Technology, Madurai, India

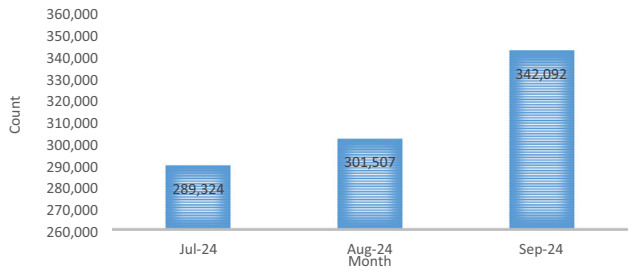


Fig. 1 Web phishing attacks trend Q3,2024

Figure 2 illustrates the most targeted sectors affected by phishing attacks in Q3, 2024. The social media industry is the most impacted at 30.5%, followed by webmail, at 21.20%, and the financial sector, at 13%.

The daily generation of new phishing URLs poses substantial challenges. Several existing studies have proposed using machine learning (ML) techniques to identify phishing URLs [9]. However, these techniques are associated with several limitations. The one significant challenge is the manual feature extraction, which is time-consuming, prone to errors, and requires the involvement of expert professionals. Phishing URL detection has recently gained popularity with the adoption of deep learning (Dl) based models, including Convolutional Neural Networks (CNN), Long short-term memory Networks (LSTM), and Recurrent Neural Networks (RNN) [10]. These techniques can identify features from unprocessed data, reduce the dependency on feature engineering and manual intervention, and address the limitations of lexical-based feature extraction. Furthermore, deep learning models are scalable, capable of handling large data volumes and can improve e classification performance in complex environments [11]. However, publicly available benchmark datasets are often not updated regularly, while attack patterns continue to evolve [12–14]. Blacklisting is the most popular and quick detection method, but it has limitations in identifying newly developed phishing websites [15]. This study aims to address these gaps by creating a dataset and developing an optimized model for detecting web phishing attacks through a detailed experimental evaluation. We present a model based on three experiments to enhance performance and reduce false positives and losses to extend the current studies. This study addresses the following research question (RQ).

RQ1. What are the existing studies, methods, and publicly available datasets for web phishing detection? What are the techniques to create phishing datasets?

RQ2. How can a deep learning classifier detect phishing attacks?

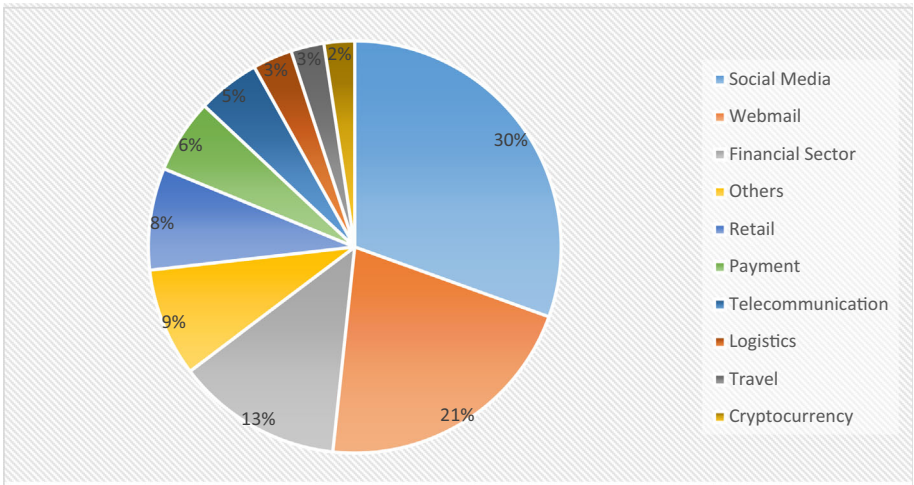
RQ3. What is the significance of optimizing the deep learning-based model, and how does it impact performance?

This study explores existing datasets and methods for detecting web phishing attacks. It proposes a model that includes feature engineering and deep learning-based optimization to enhance performance and address the research questions.

The significant contributions of this study are as follows:

1. To summarize existing techniques and solutions for phishing attacks, enabling organizations and users to effectively understand recent trends.
2. To create a web phishing dataset based on samples from trusted sources. This process will support the researchers in developing phishing datasets to explore unseen features and extend existing studies.
3. To propose a novel EGSO-CNN model for detecting phishing attacks using deep learning methods and an optimization technique to enhance performance while reducing false positives and error rates. The study employed StandardScaler for preprocessing to remove unwanted

Fig. 2 Most targeted sectors



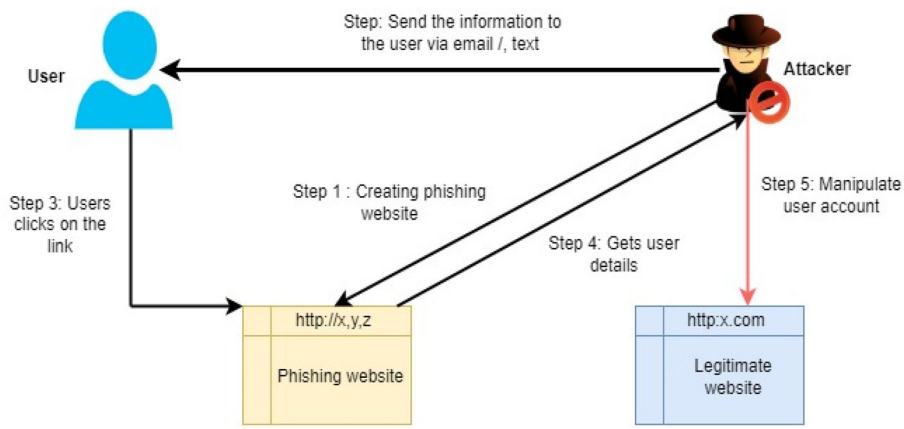


Fig. 3 Phishing life cycle

null values from the dataset. The features are extracted using Variational Autoencoders (VAE).

4. To validate the proposed model using a different dataset and compare it with the existing related studies.

The remaining paper is formulated as follows: Sect. 2 demonstrates the study background, existing dataset, and phishing detection methods. Section 3 depicts the proposed methodology. Section 4 illustrates comprehensive experimentation. Section 5 emphasizes the study's contribution to IS knowledge, practical implications, limitations, and future research directions. Finally, the paper is concluded in Sect. 6.

2 Background and related work

This section discusses the study background, existing datasets, and ML and DL-based studies on web phishing detection.

2.1 Background of the study

The phishing phenomenon was initially documented in 1996 in the hacker newsgroup [16]. The term "phishing" originated from the word "fishing," with the substitution of the letter "f" with "ph" [17]. A user account with compromised credentials is generally called phishing, and the person who successfully obtains the stolen credentials is known as the phisher. A phishing website is designed to emulate a real website and steal users' sensitive information to exploit it [18]. Attackers commonly manipulate a target webpage's hostname and path components to create a phishing UR [19]. The term URL (Uniform Resource Locator) refers to a web address that identifies the location of a resource. A web resource can encompass various formats, including web pages, text files, emails, images, and database access [20]. A URL consists of two primary elements: the protocol identifier and the resource path. Figure 3 shows the phishing life cycle. First, an attacker creates a phishing website similar to a legitimate website. The attackers manipulate URL elements, such as the domain name and network resource directory, by introducing spelling errors or using similar alphabetic characters to create a clone version of a legitimate website. For example, the URL "https://aimazon.az-z7acyu3z0y10.abc/m" replicates the appearance and functionality of the legitimate website "https://www.amazon.com." When users access fake websites, attackers capture sensitive information for exploitation.

2.2 Existing dataset

The quality of the dataset plays a crucial role in the performance of the detection model. We review the publicly available web phishing datasets, which are the most popular, widely used, and comprehensive. Several web phishing detection studies often use the PhishTank [21] dataset. This dataset encompasses approximately seven million records related to phishing URLs and is developed as a benchmark for ML and DL-based web phishing detection systems. Alexa [22] is an online platform developed by Amazon that offers a range of analytical tools for evaluating website performance. Its ranking system utilizes probability to rank millions of benign website URLs. The OpenPhish [23] dataset contains URLs associated with phishing activities and provides metadata, including page content and IP address. The Kaggle [24] offers a web phishing detection dataset containing 87 extracted features.

The Mendeley [25] published in 2021, this dataset consists of authentic and phishing website samples. Each instance contains the URL and the corresponding HTML page. The PhishStorm [26] is a publicly available phishing dataset consisting of a CSV file. The domain column provides a unique identifier for each entry. The label column provides the domain entry status: 1 for legitimate and 1 for phishing. Table 1 summarizes the publicly available web phishing datasets.

Table 1 Existing publicly available datasets

Name/Reference/last update	Phishing size	Legal size	Link
PhishTank [21], 2017	4.8 K	1.2 K	https://phishtank.org/developer_info.php
Alexa [22], 2023	2.5 K	2.5 K	https://www.alexa.com/
OpenFish [23], 2023	6.2 K	6.2 K	https://openphish.com/phishing_database.html
Kaggle [24], 2021	114 K	392 K	https://www.kaggle.com/datasets/
Mendeley [25], 2021	50 K	30 K	https://data.mendeley.com/datasets/n96ncsr5g4/1
PhishStorm [26], 2014	48 K	48 K	https://research.aalto.fi/en/datasets/phishstorm-phishing-legitimate-url-dataset

2.3 ML-based phishing detection

Machine learning usage has significantly increased in various domains over the last decades. Ahammad et al. [27] presented a study on web phishing detection using five ML classifiers. The study employed the Phish Tank dataset of 3000 URLs. The LightGBM classifier achieved an accuracy of 89.5%, but the study considered the limited features. Alani et al. [28] offered a comprehensive analysis of phishing URL detection employing ML techniques. In the feature selection stage, the limited necessary features are eliminated through recursive feature elimination. The dataset was created by collecting data from Phish Tank and Alexa datasets. The random forest classifier demonstrated the highest accuracy of 97.5% but, the proposed model was evaluated in the cloud environment. Korkmaz et al. [29] presented a study on web phishing detection ML-based techniques using eight different classifiers. The authors utilized different evaluation metrics, i.e., accuracy, TPR, FPR, TNR, FNR, precision, f1-score, and time. Barraclough et al. [30] proposed a study on intelligent online cyber phishing attack detection, integrated web content, and heuristic-based approaches using different techniques. The authors demonstrated an accuracy of 99.33% using the PART classifier.

Jain et al. [31] presented an effective approach to detecting web phishing pages using ML classifiers. The author employed effective feature selection techniques to detect zero-day attacks and used the Alexa dataset. The proposed model was evaluated using five different ML classifiers, and the random forest classifier achieved the highest accuracy of 93.85%. Sánchez-Paniagua et al. [32] proposed a model based on ML techniques for phishing detection and utilized handcrafted feature selection. Three sets are employed: legitimate home, login, and phishing. The authors validated the model using eight ML classifiers. The LightGBM yielded the highest accuracy of 94.67% using PIU-60 k datasets, and XGBoost accomplished an accuracy of 93.22%. Jha et al. [33] proposed a study identifying and distinguishing a web phishing website. Using logistic regression, the authors exhibited the highest accuracy of 98% compared to the ML classifiers employed. Karim et al. [34] studied phishing URL detection using hybrid ML techniques. The study used voting techniques that combined multiple ML classifiers, and a canopy-based feature selection technique was utilized. The author achieved an accuracy of 98.12%, a precision of 97.31%, and an f1-score of 95.89%. Shirazi et al. [35] presented a study on phishing detection based on synthetic data generation using adversarial autoencoder and WGAN-based (Wasserstein Generative Adversarial Network) techniques. The authors achieved an accuracy between 97 and 98% in detecting phishing URLs using different ML-based classifiers. The study considered limited features and did not consider the data-cleaning process. Pandey et al. [36] proposed a model for web phishing detection based on the web page's predominant color. The authors employed five ML classifiers, and the random forest classifier demonstrated the highest accuracy of 99.13%. The study considered a limited number of features and did not consider the error rate.

2.4 DL-based phishing detection

Smadi et al. [37] presented a neural network model for zero-day phishing email detection. The authors employed reinforcement learning to enhance the performance and reduce the false positive rates. The proposed model demonstrated an accuracy of 98.63%. Yang et al. [38] proposed a multidimensional feature phishing detection technique based on quick detection using the DL technique. The character sequence characteristics from URLs are retrieved and employed by DL for categorization. CNN is employed to extract local correlation information. The author demonstrated an accuracy of 98.88% but did not focus on the error rate and processing time. Zhu et al. [39] proposed a model for phishing detection using a neural network and decision tree with optimum features. The authors demonstrated an accuracy of 97.80% and a precision of 97.76%.

Tang et al. [40] presented a novel framework for web-based phishing detection using RNN-GRU. The author used four different data sets for validation and accomplished an accuracy of 99.18%, reducing the calculation time and false alarm rates. Al-Ahmadi et al. [41] proposed a phishing detection model based on LSTM, CNN, and GAN. The LSTM

generates synthetic phishing URLs, and CNN determines whether real or fake. The authors presented a precision of 98.02% and an accuracy of 97.58% without depending on third-party services.

Almousa et al. [42] presented a model for web phishing detection using a systematic approach, CNN, LSTM, and a Fully Connected Deep Neural Network (FC-DNN) with parameter optimization. The study employed four publicly available datasets to validate the model and demonstrated an accuracy of 97.37%. Hussain et al. [43] presented a model named CNN-Fusion, a practical and lightweight web phishing URL detection based on character-level CNN. The CNN layers are regularized using Dropout, Batch normalization, and SpatialDropout1D. Prabakaran et al. [44] presented a model integrating DL-based Variational Autoencoders (VAE) and DNN. The features are automatically extracted using VAE. The authors validated the model using two datasets and demonstrated an accuracy of 97.45%, but the study exhibits a relatively high false positive rate of 2.19%. Kumar et al. [45] proposed a model based on the binary bat and neural network for identifying phishing attacks. The proposed model used Adam's optimization techniques and yielded an accuracy of 94.8%. Opera et al. [46] presented a model of phishing detection using a trained DNN to analyze embedded raw URLs and HTML contents. The authors demonstrated an accuracy of 98.1% with minimum false positives and false negatives. Table 2 summarizes the existing DL-based web phishing detection studies.

One of the primary challenges identified in existing ML-based phishing detection studies is their reliance on manual feature engineering and difficulty handling a large dataset. Attackers may understand the explicit features on which the model has been trained and bypass the security mechanism. The DL-based model can address the large amount of data in complex environments with high processing speed. However, few studies have created web phishing datasets, including multiple data sources, and employed processing and optimization techniques. Although prior studies have explored DL-based model methods to classify raw HTML strings, enhancements are required to achieve enhanced accuracy and reduced error rates in classification.

The study aims to address these gaps by creating a web phishing dataset based on updated samples from trusted sources. The study used a comprehensive pre-processing technique to remove unwanted null values from the dataset. Three deep learning classifiers, CNN, LSTM, and GRU, have been employed to build the model. The proposed model with optimization techniques is evaluated based on three experimentations and validated with a different dataset, emphasizing practical applicability.

3 Research methodology

The proposed model comprises a data module, preprocessing, feature extraction, deep learning classifiers, optimization, performance analysis and evaluation, as demonstrated in Fig. 4. The data collection module gather data from various sources and construct a primary dataset. The dataset undergoes rigorous cleaning and processing followed by extraction of essential features. The study employed deep learning-based classifiers with optimization techniques to build the model for performance evaluation in detecting phishing attacks.

3.1 Data collection

The significant challenge in web phishing attack detection is the availability of the dataset. Benchmark datasets allow researchers to effectively compare the performance of their models. However, data quality plays a critical role in determining a model's accuracy and reliability. A major issue is the absence of a comprehensive and up-to-date labeled training dataset, which hampers the development of effective ML and DL-based web phishing detection systems. The continuous evolution of phishing URLs and inclusion of expired or outdated samples in existing datasets can negatively affect a model's accuracy [47]. Biased detection can result from unequal distributions of phishing versus legitimate URLs. Since phishing techniques constantly evolving, periodically updated datasets are critical [48]. More studies are needed to create different web phishing datasets, enabling researchers to assess their models' performance more efficiently. To address these challenges, this study aims to create the primary dataset based on the samples collected from trusted sources, namely Kaggle [24], Alexa [22], and PhishStorm [26]. Figure 5 presents the details of the data collection module.

3.2 Data construction and description

As summarized in Table 3, the primary dataset is created by collecting samples from three trusted sources: Kaggle, Alexa, and PhishStrom.

The data construction process is illustrated in Fig. 6, outlining each stage from raw data acquisition to final dataset creation. The collected data are pre-processed to ensure consistency and remove invalid or incomplete entries. URLs are split into key components such as protocol, domain, subdomain, and path to extract lexical and statistical features. Features are categorized into three groups : URL-based, content-based, and external-based features.

Table 2 Summary of existing DL-based web phishing detection studies

Reference	Methods	Datasets	Approaches	Strengths	Limitations
[38]	A multidimensional feature phishing detection methodology (MFPD)	Phish Tank and dmoztools.net	Dynamic category decision algorithm, CNN, LSTM	Dynamic detection using CNN and LSTM using comprehensive multidimensional feature detection	Not considered feature extraction of webpages and texts using DL
[39]	Decision Tree and feature-based ANN	Data was collected from 10 datasets based on the feature	Artificial Neural Networks (ANN)	High interpretability and decision-making using feature-based learning	Optimum feature selection not considered
[40]	Deep learning-based framework for phishing detection	PhishStorm, PhishTank, ISCX-URL2016, and Kaggle	RNN-GRU	A novel DL approach that combines RNN-GRU and evaluation with different scenarios	Short URLs are not supported
[42]	Optimization technique of deep learning	Lin Tan dataset, 2018, Phish Tank and Alexa	LSTM, CNN	Focused on optimization techniques to enhance phishing detection capabilities	Feature extraction is not considered
[43]	CNN-Fusion, lightweight, lightweight, character-based CNN	5 publicly available datasets	CNN	Phishing detection using a lightweight CNN model that extracts character-level features effectively	Only considered character level other features are not considered
[44]	Features are extracted using VAE, and raw features are automatically extracted	ISCX-URL-2016 dataset and Kaggle dataset	DNN (Deep Neural Networks)	Improved model scalability by automatically extracting features using VAE	False Positive rates are high
[45]	Adam optimizer algorithm	Kaggle dataset	SI-BBA	Improved model stability by fine-tuning using Adam optimization	Limited dataset
[46]	Analyzing raw URLs and HTML contents using DL techniques	Alexa dataset	CNN	Enhanced phishing detection using richer data characteristics, incorporating embedded raw URLs and HTML texts	Limited features used and manipulation of web page not considered

Vectorization enables the machines to comprehend textual elements by converting them into meaningful numerical representations. The Term Frequency-Inverse Document Frequency (TF-IDF) transforms the textual features into numerical vectors, capturing the relative importance of each term across the dataset. The feature vectors are scaled using the StandardScaler method to normalize values and reduce bias. Variational Autoencoders are employed to enhance feature representation further and reduce dimensionality. Subsequently, the final primary corpus included a balanced dataset of 27,534 samples, i.e., 13,767 phishing and 13,767 legitimate URLs, ensuring model stability. To validate the proposed model, the study employs PhishTank [21] as a secondary dataset, containing 20,004 samples: 10,002 phishing and 10,002 legitimate URLs. The dataset employed all the elements of an HTML record, mainly text, hyperlinks, lists, etc., and all the URL details to train the model. The URL prefixes, such as http:// and https://, are dropped to control irregular consequences on various URL datasets. The description of the dataset is illustrated in Table 4.

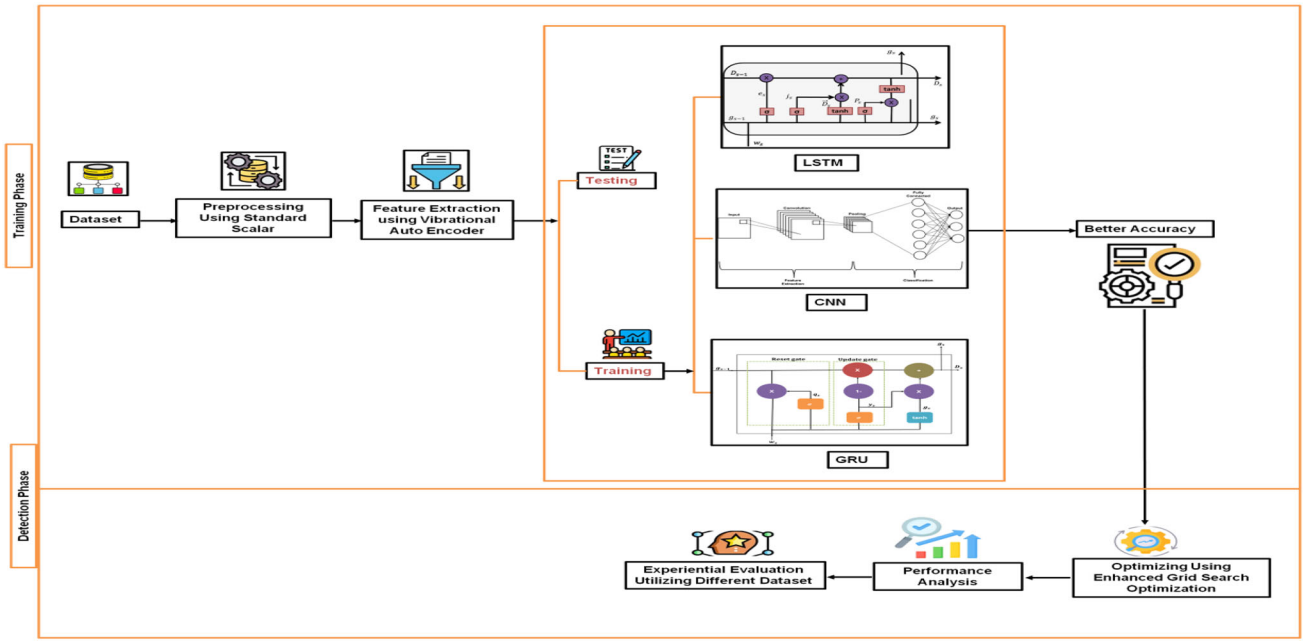


Fig. 4 Proposed model

Fig. 5 Data collection module

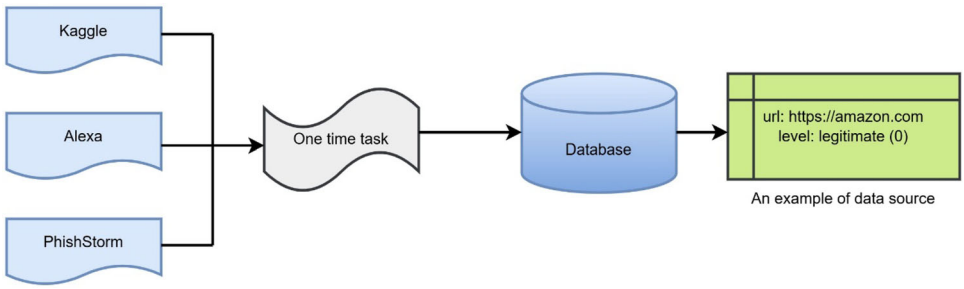


Table 3 Description of the data samples

Data Source	Legitimate URLs	Phishing URLs
Kaggle	5715	5715
Alexa	6425	6121
PhishStorm	5241	5126

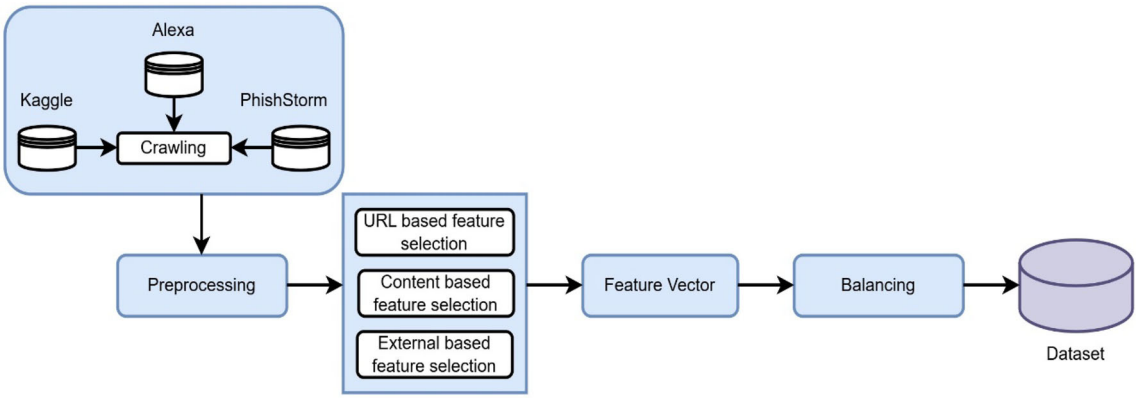


Fig. 6 Data construction process

Table 4 Description of the dataset

Name	Description	Sample
Url	URL	https://paypal.mhm.rest/payment/
Label	0: legitimate; 1: phishing	1
Source	Source of Data	PhishStorm
Url length	Length of URL	29
Date_creation	creation date	49

3.3 Data preprocessing and feature extraction

The StandardScalar technique scales to unit variance and removes the mean, ensuring consistency and eliminating bias that can impair the model's performance. It standardizes data fields by scaling their mean and standard deviation to 0 and 1 [49]. By subtracting the starting value from the converted value and dividing it by the standard deviation, the transformed value can be produced using Eq. 1.

$$z = \frac{x - \mu}{\sigma} \tag{1}$$

where x represents the feature's original value, z represents its modified value, μ represents the mean, and σ represents the standard deviation. The flow of the Standard Scaler employed in this study is presented in Fig. 7.

Algorithm 1 represents the pseudocode code employed for preprocessing.

Algorithm 1 Pseudocode of Standard Scaler

Input: Dataset with 'n' samples and 'm' features (attributes).

Output: Standardized dataset.

Step 1: Compute the mean (μ) *and standard deviation* (σ) *for each feature across all the samples in the dataset;*

Step 2:For each feature i *(where* $i = 1$ *to* n*):*

Step 3: Calculate the mean: $\mu_i = sum(X_{ji})/m$*, where* X_{ji} *is the value of feature* i *for sample* j*.*

Step 4: Calculate the standard deviation: $\sigma_i = sqrt(sum\left(\left(X_{ji} - \mu_i\right)^2/m\right)$*.*

Step 5: For each sample j *(where* $j = 1$ *to* m*) and each feature* i*:*

Step 6: Calculate the standardized value: $z_{ji} = \left(X_{ji} - \mu_i\right)/\sigma_i$*, where* X_{ji} *is the original value of feature* i*,* μ_i *is the mean of feature 'i, and* σ_i *is the standard deviation of feature 'i.*

Step 7: Return the standardized dataset, where each value X_{ji} *in the original dataset has been replaced by the corresponding* z_{ji} *calculated.*

The Term Frequency-Inverse Document Frequency (TF-IDF) algorithm is employed for feature extraction and vectorization [50]. The number of times a word d occurs in a document t divided by the total number of words in the document, is its term frequency (TF), as presented using Eq. 2.

$$\mathrm{TF}(t, d) = \frac{\text{Count of term } t \text{ in document } d}{\text{Total number of terms in document } d} \tag{2}$$

where t is the term, d is the document.

The importance of a phrase in all of the corpus's texts is measured by its inverse document frequency (IDF), as shown in Eq. 3.

$$\mathrm{IDF}(t) = \log\left(\frac{N}{1 + \mathrm{DF}(t)}\right) \tag{3}$$

where N is the total number of documents in the corpus, and DF (t) is the document frequency of the term t.

The TF-IDF scores are presented as a vector for each phrase in a document. A vector of size V is used to represent each document in a corpus of N documents with a vocabulary of V unique words. Each element in the vector represents the TF-IDF score for a particular vocabulary, as shown in Eq. 4. The TF-IDF process employed in this study is presented in Fig. 8.

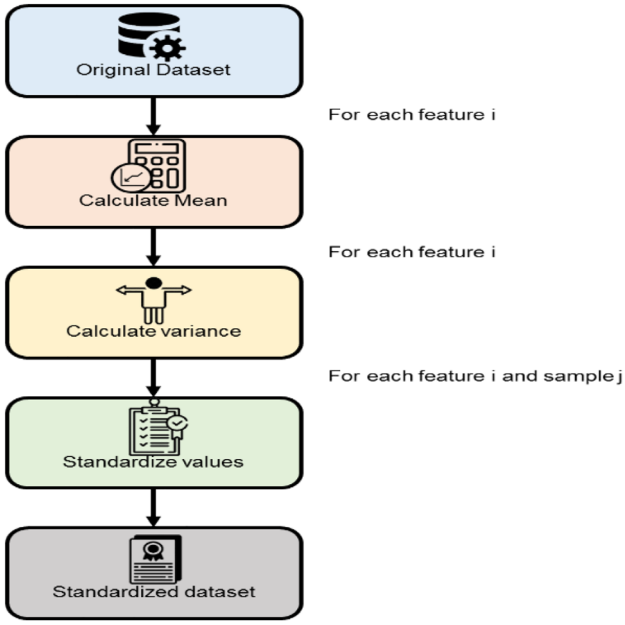


Fig. 7 Process of the standard scaler

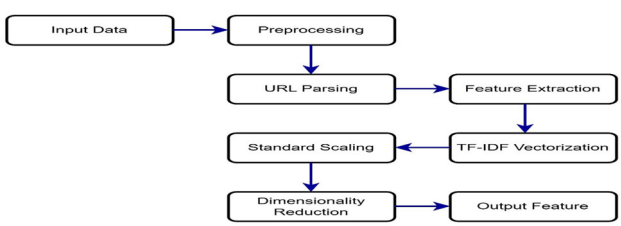


Fig. 8 TF-IDF process

$$\vec{d} = [\text{TF - IDF}(t_1, d), \text{TF - IDF}(t_2, d), \ldots, \text{TF - IDF}(t_v, d)] \quad (4)$$

Feature generation creates new features from existing ones that can benefit the model. Feature generation can identify in-depth patterns in the data. For example, URL components can generate additional features such as URL entropy, the number of digits or special characters, and subdomain length. The Variational Autoencoder (VAE) enhances this process by encoding inputs into a probability distribution within the latent space. This probabilistic method permits VAEs to sample from the latent distribution, facilitating the creation of diverse data instances and enhancing the modeling of data variability [51]. It is useful for automatic feature generation by learning latent representations that capture the core structure of URLs. Unlike manual feature engineering, VAEs generate meaningful patterns less prone to human error. These generated features are combined with extracted ones to improve the classifier's performance and reduce overfitting. The feature extraction technique filters out certain inbuilt URL aspects to improve the classifier's performance. Decreasing the dimensionality of the input URL may reduce the time overhead connected with the classifier's training. The decoder utilizes the latent variable to reassemble the source data after the encoder has transformed it into a latent variable.

Figure 9 depicts the VAE architecture and aims to reduce and build a network that reassembles its input data w as nearly as possible to $\hat{w}$ using Eq. 5.

$$K(w, \hat{w}) = w - \hat{w} \quad (5)$$

The VAE is a randomized computational framework where the probability parameters $r(y|w, \phi)$ and $o(w|y, \phi)$ determine the encoder and decoder. Specifically,$o(w|y, \theta)$ is the probability of given y, and $r(y|w, \phi)$ is the approximate posterior, known as the adversarial encoder. After completing the required training, the encoder can map the input information w to the latent space similar to the posterior distribution. The y is distributed according to the Gaussian theory based on the average $\hat{y}$ and deviation σ^2 values. The value of $\hat{y}$ can be utilized as an instance from $M(\mu, \sigma^2)$, and the decoder would be tuned to produce the outcome. The probability margins for each data point are presented using Eq. 6.

$$\log o_\theta(w) = \sum_{j=1}^{m} \log o_\theta\left(w^{(j)}\right) \quad (6)$$

where θ refers to the variable of the decoder system and m denotes the quantity of data. The data point $w^{(j)}$ can be represented using Eq. 7.

$$\log o_\theta\left(w^{(j)}\right) = LK\left(r\left(y|w^{(j)}, \phi\right), o\left(y|w^{(j)}, \phi\right)\right) + K\left(\theta, \phi; w^{(j)}\right) \quad (7)$$

where $o(y|w, \phi)$ stands for the actual posterior distribution. The Kullback-Lieber (KL) diverging calculates the distance between two distributes, as shown in Eq. 8.

$$K\left(\theta, \phi; w^{(j)}\right) = -LK\left(r\left(y|w^{(j)}, \phi\right), o(y, \theta)\right) + F_r\left(y|w^{(j)}, \phi\right)\left[\log o(w^{(j)}|y, \theta\right] \quad (8)$$

Since KL function is non-negative can be represented using Eq. 9.

$$\log o_\theta\left(w^{(j)}\right) \geq -LK\left(r\left(y|w^{(j)}, \phi\right), o(y, \theta)\right) + F_r\left(y|w^{(j)}, \phi\right)\left[\log o(w^{(j)}|y, \theta\right] \quad (9)$$

The first element on the correct side of disparities is the KL divergence among the subsequent and prior latent parameter y estimates. The predicted reconstruction error may be considered the second term on the right side of the inequality. The KL difference can be estimated if the previous $o(y, \theta)$ is under the influence of an average distribution $M\left(\mu, \sigma^2\right)$. However, the expectation should be estimated using a Monte Carlo method for the second component. We can first sample $\hat{y}$ from an unknown distribution $r\left(y|w^{(j)}, \phi\right)$ before decoding. The sampling process derivative, which prevents the network from using gradient-based training, can be computed using Eq. 10.

$$y = \mu + \sigma t \tag{10}$$

Equation 11 represents the objective function for VAE to determine $y^{(j,i)}$ for the jth data point using the ith sample $T^{(i)}$ from the normative average dispersion. The encoding process in VAE translates the supplied data into a stochastic probability of the latent factor, which is collected from the dispersion. Algorithm 2 presents the pseudocode employed for feature extraction in this study.

3.4 CNN and enhanced grid search optimization (EGSO)

The Convolution Neural Network (CNN) approach reduces processing time by distributing the weights over a temporal dimension [52]. The CNNs are used to extract sequential data from the URL. Let u be a string of letters joined together, and H contains a group of nearby components h_s, $j = 1, \ldots K$ in a concatenation to get a matrix representation $u \rightarrow H \in \mathbb{Q}^{k \times L}$. An embedding of the form $h_s \in \mathbb{Q}^L$ represents each l-dimensional vector. The embedding vector from the manually initialized embedding matrix is used to create the three-dimensional component representation. Concatenating K components together can be presented using Eq. 12.

$$H_{1:K} = h_1 \oplus h_2 \oplus \cdots \oplus hk \tag{12}$$

where $\oplus$ presents the concatenation operator. All sequences are truncated to the same length K or filled with a value of 0. Using a convolution operator, the CNN network convolves up through this instance $H \in \mathbb{Q}^{K \times L}$. The h-length convolution creates a new feature by convolving a filter $W \in \mathbb{Q}^{l \times g}$ and a nonlinear activation e, as shown in Eq. 13.

$$K_{VAE} = \sum_{j=1}^{m}\left(\frac{1}{2}\sum_{i=1}^{I}\log o\left(w^{(j)}|y^{(j,i)}, \theta\right) - LK\left(r\left(y|w^{(j)}, \phi\right), M(0, 1)\right)\right) \tag{11}$$

Algorithm 2 Pseudocode of the VAE algorithm

Input: Dataset$W$; Parameter: θ and ϕ; Mini-batch: $batch$; Epochs: $epoch$; Rate of learning: kq

Output: Results of detection.

Step1: Set the parameters initialized

Step 2:for $L1 \in \{1, 2, \ldots, epoch\}$ do

Step 3: for $L2 \in \{1, 2, \ldots, batch\}$ do

Step 4: for each $w \in W$, do

Step 5: Determine each layer's input and output vectors

Step 6: end for

Step 7: end for

Step 8: end for

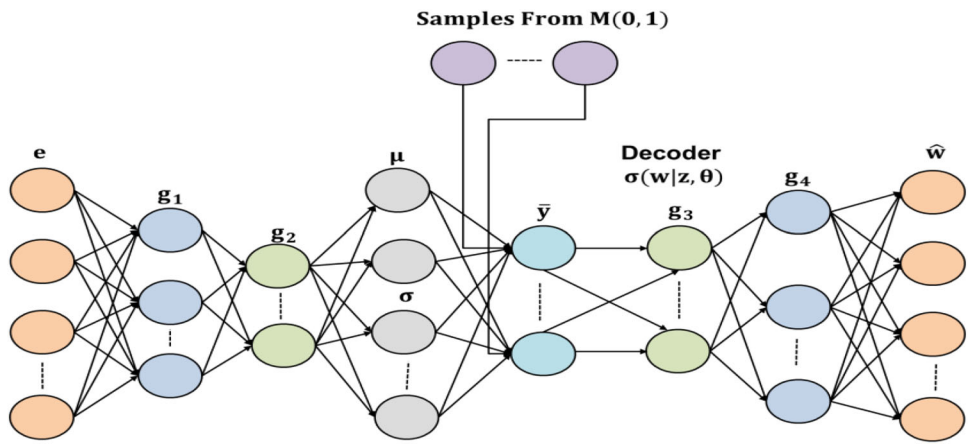
Step 11: Creating a matrix W^*

Step 12: The first matrix of features W is integrated with the newly translated feature matrix W^* to acquire the matrix $Z = \{z_j\}_{j=1}^{M}$

Step 13: Detecting phishing attempts with representation vectors Z

Step 14: Return outcomes

Fig. 9 VAE architecture



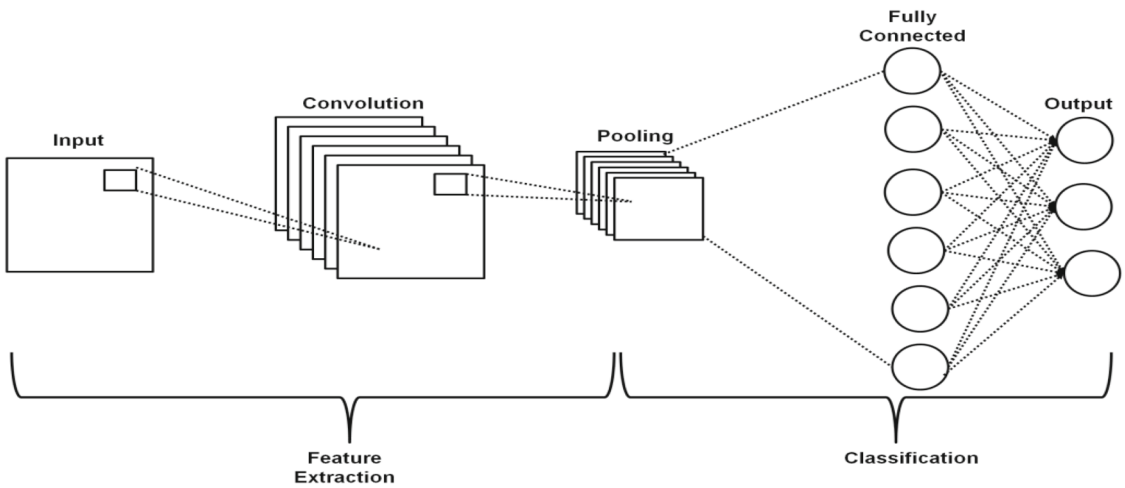


Fig. 10 CNN Architecture

$$z_j = e\left(W \oplus H_{j:j+g-1} + a_j\right) \tag{13}$$

where a_j is the bias.

Each g-length segment of its inputs, separated by a preset stride value, is applied to the W filter as part of this convolution layer's output. After concatenating these outputs, the Z can be presented using Eq. 14.

$$\Upsilon = \left[z_1, z_2, \ldots, z_{K-g+1}\right] \tag{14}$$

The pooling step reduces the feature's dimension, emphasizes the important features, and trains the model. The CNN can benefit from the temporal connection of length h in its input by using a filter W to convolve each segment of length g. The CNN comprises many filters, each with a distinct length (g) and several filters in each group, as shown in Fig. 10. Algorithm 3 presents the pseudocode for CNN used in this study.

Algorithm 3 Pseudocode for CNN

```
Input: URL u, embedding matrix E, filter W, bias a
Output: Feature map Y
Embed URL u into matrix representation H
H = []
for s in range(1, K + 1):
  h_s = Embed(u[s])
  H.append(h_s)
Concatenate all components h_s to form H (K x L matrix)
H = Concatenate(H)
Convolve H with filter W using a sliding window approach
Z = []
for j in range(1, K − g + 1):
    Get segment of length g from H (H_j)
  H_j = H[j:j + g − 1]
  Apply filter W to the segment and add bias a_j
  z_j = ReLU(Convolve(W, H_j) + a_j)
  Z.append(z_j)
Concatenate all z_j to form the feature map Y.
Y = Concatenate(Z)
Pooling (optional) should be applied to reduce dimensionality and retain important features.
Y_pooled = Pooling(Y)
end
```

The enhanced grid search optimization (EGSO) method determines the fusion of these hyperparameters. Due to the many hyperparameters and different levels, it has a high computation cost. However, adding cross-validation methods to the grid search approach can address overfitting issues [53]. Let Z be the intended algorithm using l parameters to be configured; the parameter θ_j constitute the interval's value $\left[w_j z_j\right]$ in the search space parameter $\Theta = [w_1 z_1] \times \cdots \times [w_l z_l].G : \Theta$ is changed to a function for measuring performance that gives a score in numbers to θ. Cross-validation H calculates the error, and Θ contains seven factors: $m_{hiddenlayer}$, m_{neuron}, m_{epochs}, $m_{optimization}$, $batch_{size}$, $learning_{rate}$ and $\lambda_c(dropout)$. Figure 11a depicts a grid layout with the 3 × 3 parameter set. The grid patterns are illustrated in Fig. 11b.

Let Λ be the search space and L be an implicit dimension. A function as the configuration variables' indexed set L. For each variable's collection of values, a grid search should find the best value of $\left(K^1, K^2, \ldots, K^L\right)$. The study incorporates the CNN approach with grid search optimization. The proposed architecture consists of several layers: the input dataset, grid search CNN, max pooling layer, fully connected NN, convolution layer dropout, and prediction, as demonstrated in Fig. 12.

Grid search has several disadvantages, i.e., the curse of dimensionality and high computational cost. The cross-validation increases the possibility of overfitting. The proposed method is efficient in employing optimization techniques in complex environments. It reduces the search space using heuristic techniques and incorporates cross-validation to avoid overfitting. It further refines the search space to increase computational efficiency using hyperparameter settings and early stopping to avoid excessive iterations. The procedure of pooling data involves determining the highest value on each feature map. Dropout addresses the overfitting issues. Algorithm 4 demonstrates the pseudocode employed in this study.

Algorithm 4 The pseudocode of EGSO-CNN

Input: *Dataset, where* Z *is a target,* W *is variable, and* m*-instruction sample with attributes; Enhanced Grid Search Optimization (EGSO); Algorithm (CNN); Hyper parameter* (Θ)

Step 1: Res $\leftarrow\{\}$;
Step 2: for $jtoMdo$
Step 3: $\theta \leftarrow select\ hyperparameter\ (EGSO,CNN,\theta)$
Step 4: $Model \leftarrow train\ (CNN,\theta,W_{train},Z_{train})$
Step 5: $Res \leftarrow eval\ (model,\ W_{test},Z_{train})$
Step 6: end for
Step 7: $\theta \leftarrow Adjust\ (\theta\ ,eval(res))$
Step 8: end for
Step 9: Output Optimized Parameters

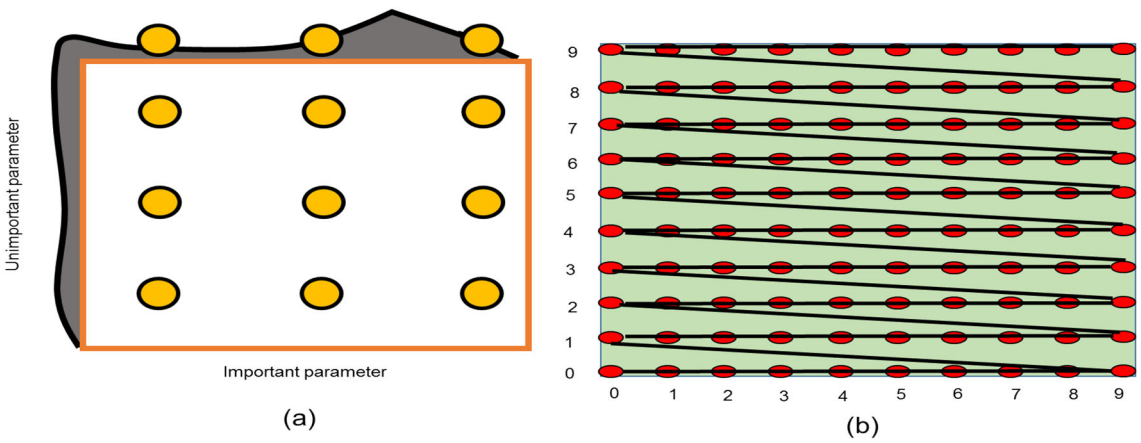


Fig. 11 Grid search **a** grid layout **b** grid patterns

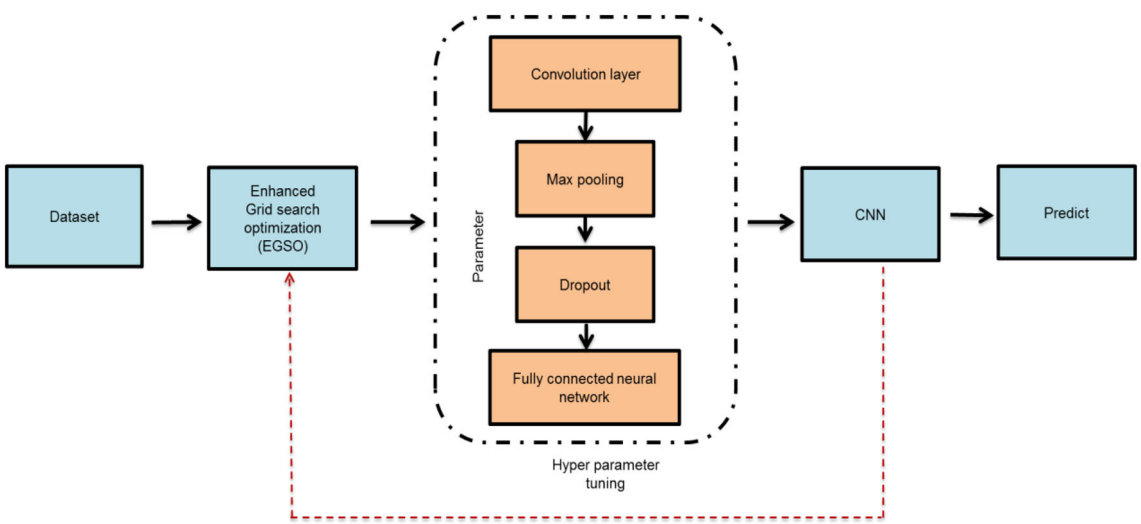


Fig. 12 The architecture of the proposed model

4 Result

We performed three experiments using Python in an Anaconda environment. The data was split 80% into training and 20% into testing. Different evaluation metrics, i.e., loss, TPR, FPR, accuracy, precision, recall, and f1-score, were employed to evaluate the proposed model's performance, as presented in Eqs. 15 to 21.

$$\text{Loss} = L(x, \hat{x}) = -\sum_j x_j \log(\hat{x}_j) \tag{15}$$

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{16}$$

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{FN}} \tag{17}$$

$$\text{Accuracy} = \frac{(\text{True Positives} + \text{True Negatives})}{\text{Total Number of Instances}} \tag{18}$$

$$\text{Precision} = \text{TP}/(\text{TP} + \text{FP}) \tag{19}$$

$$\text{Recall} = \text{TP}/(\text{TP} + \text{FN}) \tag{20}$$

$$\text{F1 - score} = 2 * (\text{Precision} * \text{Recall})/(\text{Precision} + \text{Recall}). \tag{21}$$

4.1 Experimentation 1: evaluation of three deep learning methods

In Experimentation 1, we evaluated the performance of three deep-learning classifiers: LSTM, CNN, and GRU. First, the LSTM classifier was tested and yielded an accuracy of 93.04%. Figure 13a demonstrates the accuracy of the training and validation phase, which increases over the epochs. Figure 13b displays the training vs. validation loss, decreasing over the epochs.

Next, the CNN classifier is evaluated and demonstrated an accuracy of 94.47%. Figure 14a shows the accuracy of training vs. validation. Figure 14b illustrates the training vs. validation loss, which decreases over the epochs.

Third, the GRU classifier is evaluated and achieves an accuracy of 92.95%. Figure 15a and b demonstrates the accuracy and loss in training vs. validation, respectively.

A confusion matrix is a technique used to assess the effectiveness of model categorization. It shows true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN) based on a comparison between predicted and actual labels. Figure 16 illustrates the confusion matrix of the LSTM, CNN, and GRU classifiers.

Figure 17 illustrates the three classifiers' comparative ROC and Precision-Recall (PR) curves. The AUC curve is portrayed in Fig. 17a, which shows that CNN performed better. There are small variations in classification performance between the datasets. Figure 17b depicts the precision-recall curve and indicates that CNN performed better than others.

Table 5 demonstrates the comparative outcomes of Experiment 1 and represents that CNN performed better than others.

4.2 Experiment 2: evaluation of the proposed model

In experimentation 2, we evaluated the performance of the proposed EGSO-CNN model. The hyparameter settings employed to build the model are shown in Table 6.

The proposed model enhances the performance of web phishing attacks by optimizing the number of layers, dropout rate, epochs, batch size, and learning rates. The proposed model performed optimally during experimentation with 100 epochs, after which its performance decreased, as shown in Fig. 18. The model achieved an accuracy of 99.44%, as presented in Fig. 18a. Figure 18b demonstrates the training and validation loss; a training loss of 0.2 was observed in 100 epochs.

Figure 19a demonstrates the proposed model's AUC value of 0.9902, which signifies better performance. Figure 19b exhibits that the proposed model yielded a precision-recall curve of 0.9895, symbolizing the model's capability to classify positive instances precisely.

Figure 20 presents the confusion matrix of the proposed model.

The comparative outcome of the proposed model and three selected classifiers are presented in Table 7. The proposed model outperformed the others, achieving an accuracy of 99.44%.

The dataset is further divided into subsets of 5000, 8000, and 12,000 samples to validate the proposed model's performance, as shown in Fig. 21. No significance differences were observed across four evaluation matrices, demonstrating the proposed model's consistency.

4.3 Experiment 3: validation of the proposed model

In Experiment 3, the proposed model is further validated using a secondary dataset included in this study, achieving an accuracy of 99.40%, as shown in Fig. 22a. Figure 22b presents the training and validation loss, highlighting the model's consistent performance across both datasets. The corresponding confusion matrix is exhibited in Fig. 23.

Table 8 summarizes the comparative outcomes of the proposed model in both datasets, indicating consistency.

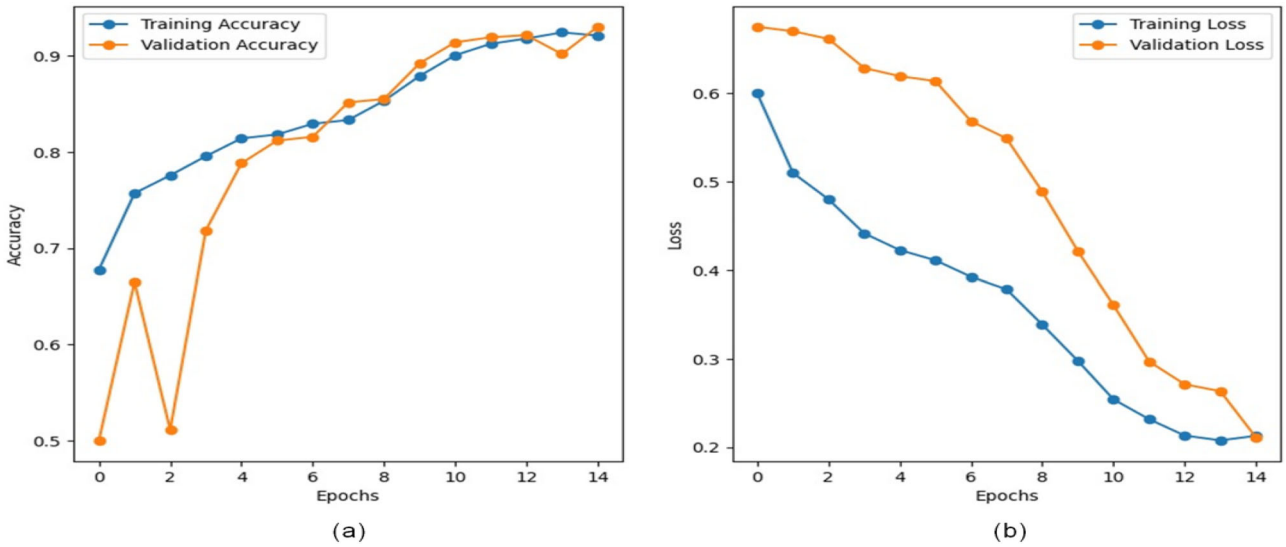


Fig. 13 LSTM performance: **a** accuracy of training versus validation, **b** loss of training versus validation

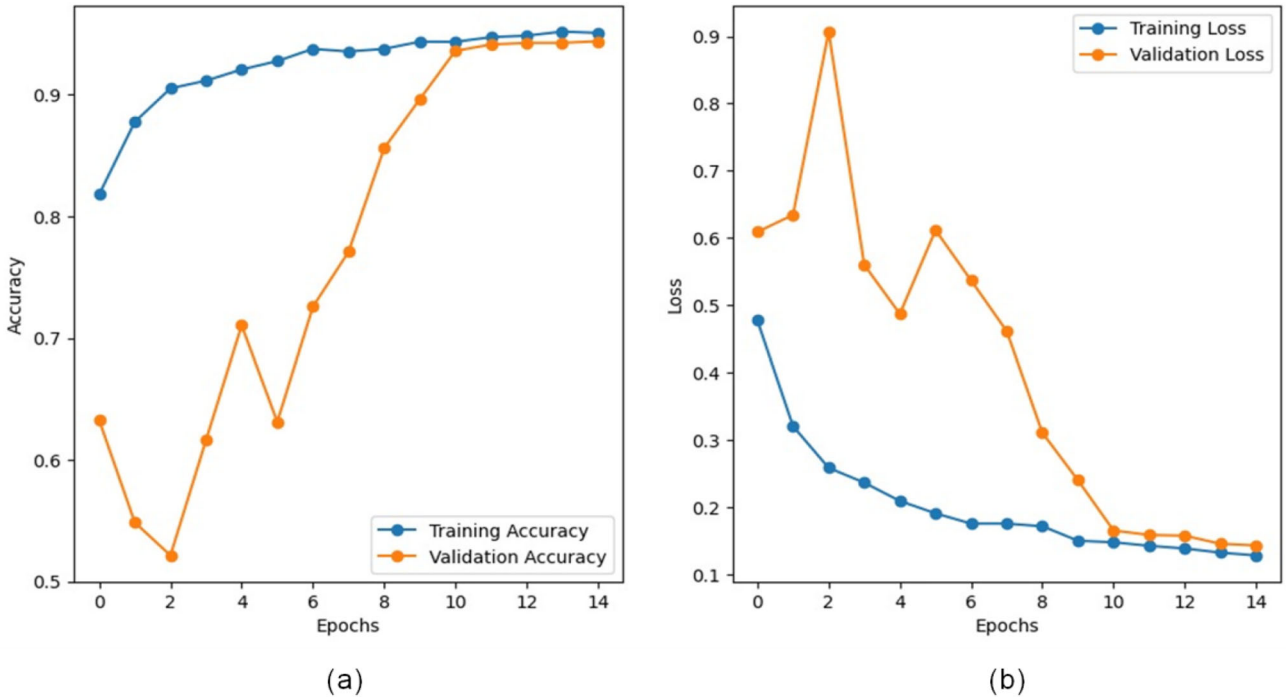


Fig. 14 CNN performance: **a** accuracy of training versus validation, **b** loss of training versus validation

4.4 Comparison with existing studies

This section compares the proposed model with existing ML and DL-based web phishing detection studies. It is important to acknowledge that the existing studies were performed under varying environments and parameter settings. The proposed model is compared with the existing five ML-based web phishing studies, i.e., RF [29], PART [30], RF [31], RF [34], and RF [36], using four evaluation parameters, as shown in Table 9.

The proposed model is compared with the existing six DL-based web phishing detection studies: DTOF-ANN [39], RNN-GRU [40], LSTM-CNN [42], VAE-DNN [44], SI-BBA [45], and DNN [46], as presented in Table 10. Figure 24 illustrates that the proposed model outperformed the others.

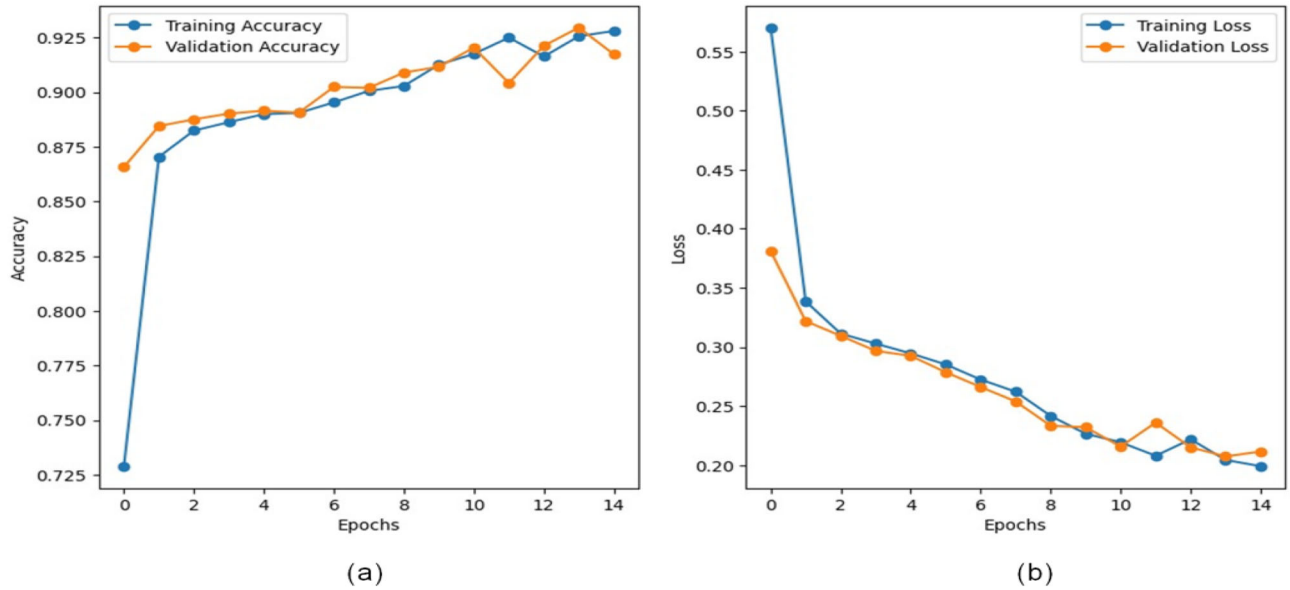


Fig. 15 GRU performance: **a** accuracy of training versus validation, **b** loss of training versus validation

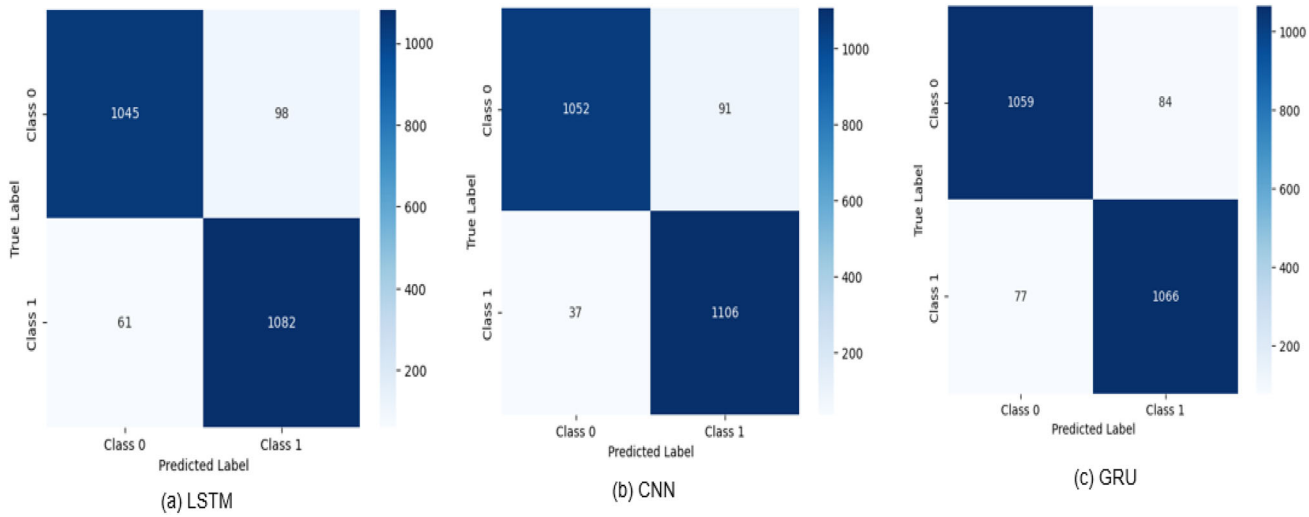


Fig. 16 Confusion matrix: **a** LSTM, **b** CNN, and **c** GRU

5 Discussion

The experimental outcomes demonstrate the practical suitability the proposed model's practical suitability for web phishing detection and address the research questions. The model's efficiency and stability were evaluated using two datasets and evaluation metrics. This study makes a significant contribution to contemporary scientific research. Additionally, the limitations of the study and directions for future research are highlighted.

5.1 Contribution to knowledge base

The study emphasized the exponential growth of web phishing attacks over the years and outlined the phishing lifecycle to raise awareness among users. It highlighted existing ML and DL-based phishing detection methods and examined publicly available web phishing datasets along with their associated challenges. We systematically created a phishing attack dataset based on samples collected from three trusted sources. Tables 1, 2, 4, and Figs. 5 and 6 address the RQ1.

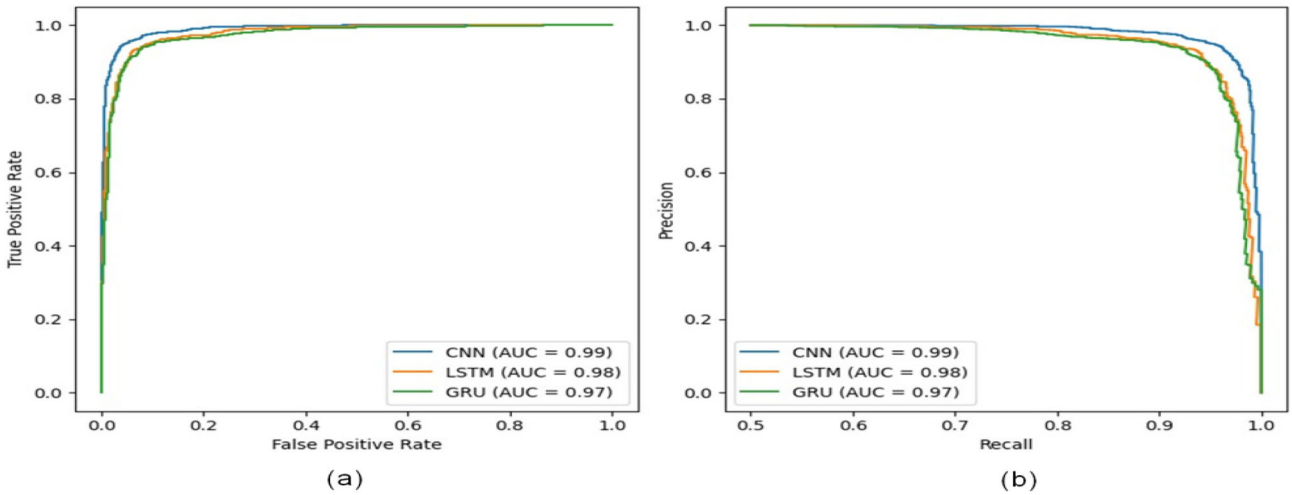


Fig. 17 Outcome analysis: **a** ROC, **b** precision-recall curve

Table 5 The outcome of three DL classifiers

Parameters	LSTM	CNN	GRU
Accuracy	93.04	94.47	92.95
Precision	92.90	93.79	90.68
Recall	92.73	96.32	94.40
F1-score	92.81	95.03	92.49

Table 6 Hyparameter settings

Parameter	Description	Value
n_estimators	Number of decision trees in the forest	200
max_depth	Maximum depth of each tree	30
min_samples_split	Minimum samples required to split a node	5
min_samples_leaf	Minimum samples at leaf node	2
filters	Number of filters in Conv1D layers	64, 128, 256
kernel_size	Size of the convolution kernel	3
dropout_rate	Dropout rate for regularization	0.5
batch_size	Number of samples per training batch	128
epochs	Number of training iterations	100
learning_rate	Learning rate for Adam optimizer	0.001

In Experimentation 1, three deep learning-based techniques, i.e., LSTM, CNN, and GRU, were employed to build the web phishing detection model. The CNN model performed best, achieving an accuracy of 94.47%, outperforming the other two models. The RQ2 is addressed in Figs. 13, 14, 15,16, and 17 and Table 5. We evaluated the proposed model in Experimentation 2 by optimizing CNN and EGSO. The optimization parameter is presented in Table 6, and the proposed model achieved an accuracy of 99.44%, as shown in Table 7. The proposed model is tested by varying the dataset size, as illustrated in Fig. 21, which demonstrated consistent performance. The proposed model is further validated with a secondary dataset in Experiment 3. It has achieved an accuracy of 99.40%, which confirms the model's consistency in detecting web phishing attacks. The model's performance was compared to the existing ML-based studies in Table 9 and DL-based studies in Table 10. The RQ3 is addressed by conducting Experiments 2 and 3 and comparing them with the existing studies.

The proposed model is independent on the dataset parameters and is suitable for real-world scenarios/applications. The

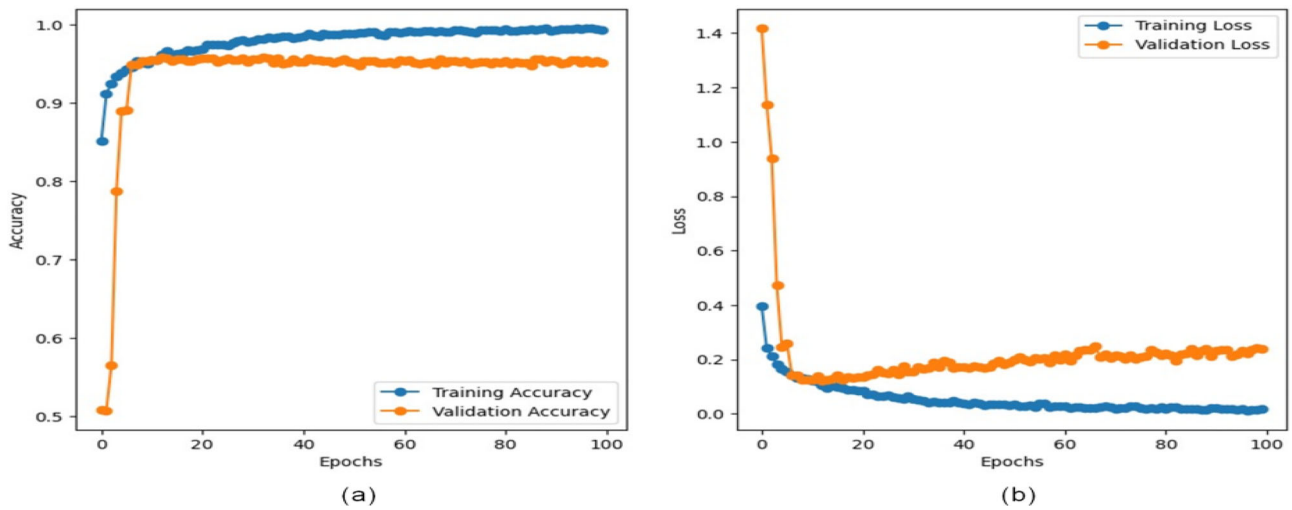


Fig. 18 Outcomes: **a** accuracy of training versus validation, **b** loss of training versus validation

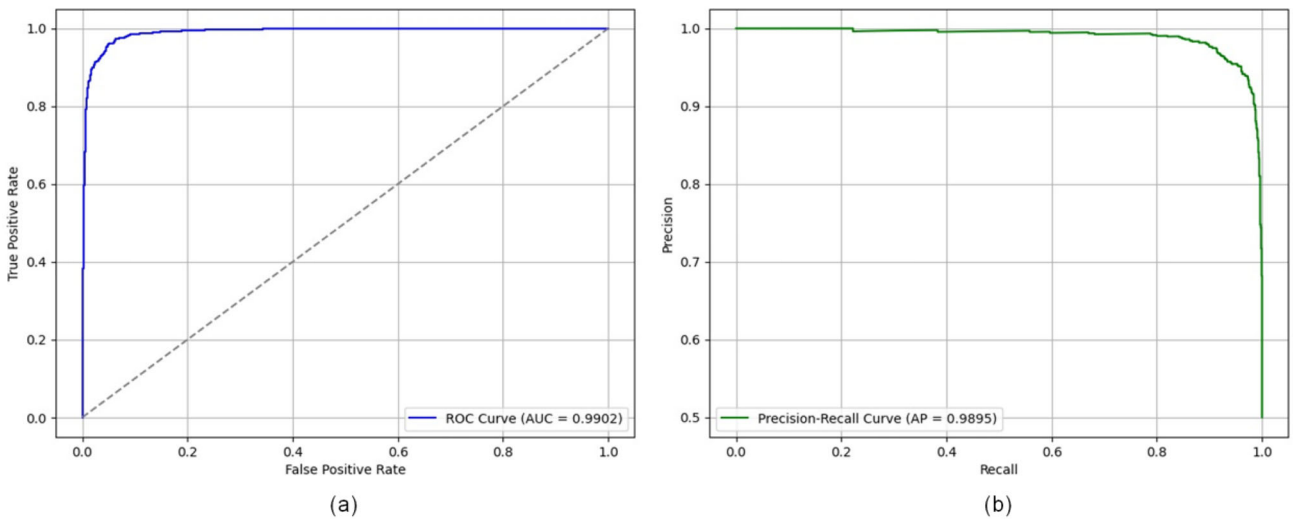


Fig. 19 Outcome of the proposed model: **a** AUC curve, **b** precision-recall curve

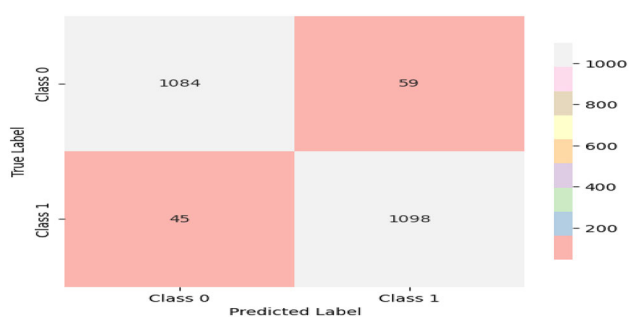


Fig. 20 Confusion matrix of the proposed model

Table 7 Comparative analysis

Parameters	LSTM	CNN	GRU	Proposed model
Accuracy	93.04	94.47	92.95	99.44
Precision	92.90	93.79	90.68	99.98
Recall	92.73	96.32	94.40	99.21
F1-score	92.81	95.03	92.49	99.32

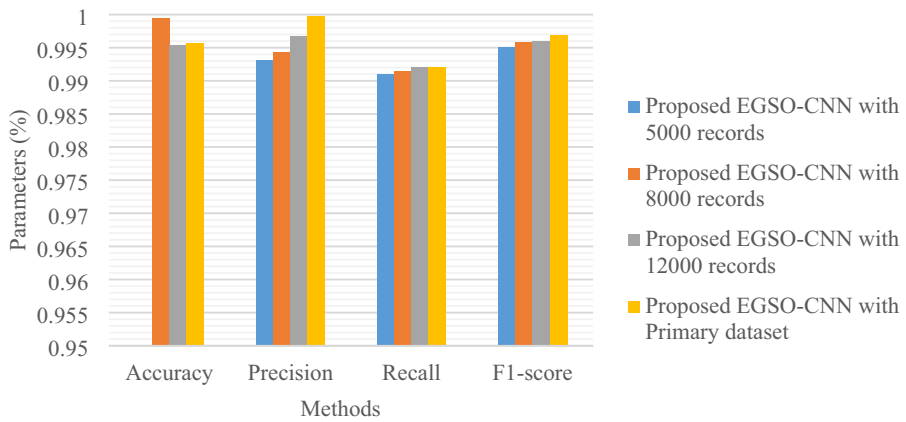


Fig. 21 The outcome of the proposed model by varying sizes of datasets

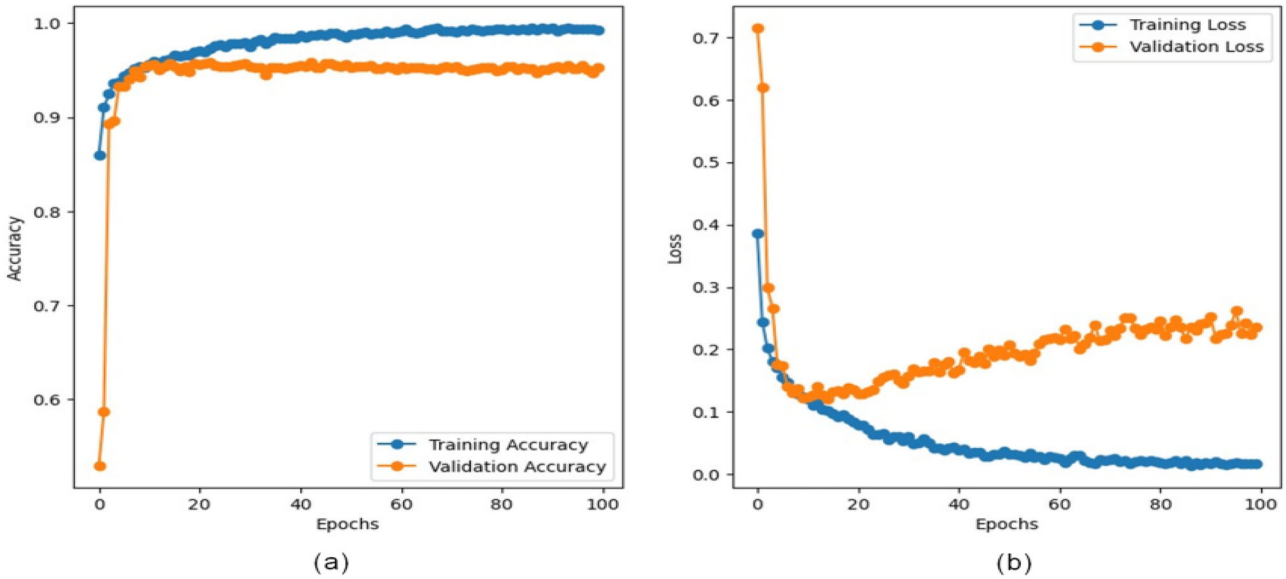


Fig. 22 Outcomes with secondary dataset: **a** accuracy of training versus validation, **b** loss of training versus validation

Table 8 Comparison of the proposed model with both the datasets

Parameters	Outcome with the primary dataset (%)	Outcome with the secondary dataset (%)
Accuracy	99.44	99.40
Precision	99.98	99.31
Recall	99.21	99.1
F1-score	99.32	99.25

Table 9 Comparative analysis with existing ML studies

Methods	Accuracy	Precision	Recall	F1-score
RF	91.26	88.18	90.12	90.53
PART	99.33	99.1	98.52	98.9
RF	93.85	94.59	94.1	94.2
RF	96.67	96.73	97.51	97.12
RF	99.13	98.43	98.98	98.96
Proposed model	99.44	99.98	99.21	99.32

Table 10 Summary of outcomes with existing DL-based studies

Methods	Accuracy	Precision	Recall	F1-score
DTOF-ANN	97.8	97.76	97.36%	97.61
RNN-GRU	99.18	98.39	98.15	98.95
LSTM-CNN	97.37	86.97	98.07	97.38
VAE-DNN	97.45	97.89	97.20	97.54
SI-BBA	94.85	94.15	94.35	94.28
DNN	98.81	98.2	98.1	98.1
Proposed model	99.44	99.98	99.21	99.32

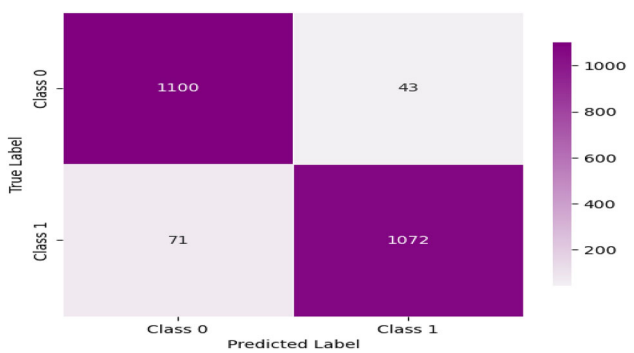


Fig. 23 Confusion matrix using a secondary dataset

different hyparameter settings and model trade-offs are utilized to assess and validate the model. The proposed model offers several benefits: it enhances prediction accuracy with low false positive rates and can be integrated into a web browser application for phishing page detection.

5.2 Practical contributions

The findings of this study extend beyond academics, offering substantial benefits to users, businesses, and the broader cybersecurity field.

1. *Enhanced Cybersecurity Standards*: Our findings demonstrate significant improvements in accuracy, reduced false positives, and lower error rates for web phishing detection with the proposed model. These advancement represents a crucial enhancement in cybersecurity techniques, especially as phishing attacks grow more in complexity and frequency. Organizations and users can trust on the system's enhanced capabilities to detect and prevent phishing attempts, thereby reducing the probability of successful attacks.
2. *Flexibility to Real-World scenarios*: The study emphasis on practical relevance ensures that the model is not merely an academic solution but a feasible tool adaptable to various dynamic environments. This flexibility is essential; the proposed model has been evaluated using diverse datasets and assessment criteria and optimized to handle large datasets in complex environments.
3. *Recommendation for Management*: The study provides valuable insights for management by demonstrating the proposed model's effectiveness in identifying phishing attacks. This model's adaptability fosters confidence in users and institutions, ensuring the system's usefulness even when faced with advanced deception techniques.

In summary, the practical implications of this study underscore that the proposed model offers considerable benefits to users and institutions in improving cybersecurity resilience and culture. The outcomes offer practical insight to guide users in a simplified way to select, implement, continuously monitor, and enhance web phishing detection systems.

5.3 Study limitations

This study has several limitations. First, phishing attack types and motivations change very fast over time. The proposed

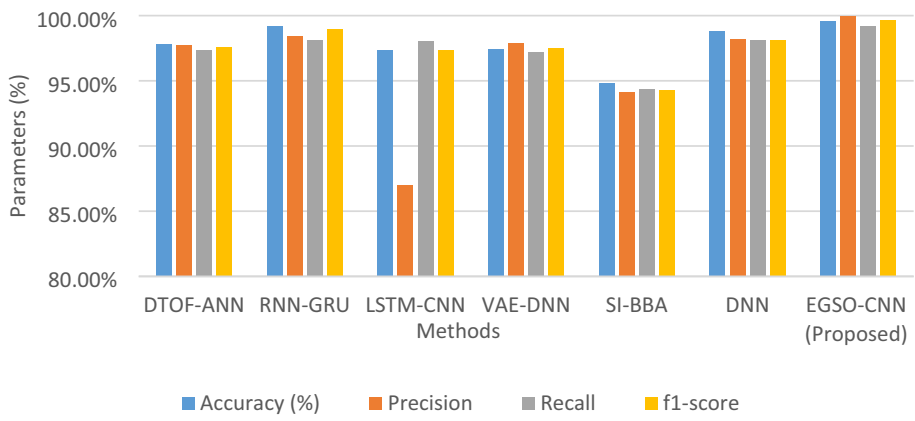


Fig. 24 Graphical comparison with existing DL-based studies

model needs to be tested and validated with more datasets. Second, it cannot detect phishing websites that substitute textual content with embedded objects like Flash and Java scripts. Separate tools are required to identify these embedded items in phishing websites. Third, this study did not consider the effectiveness of the training, test outcomes, and user feedback in phishing attacks.

5.4 Future studies

There are several future study directions. First, future research should be extended to explore phishing detection on IoT devices and mobile platforms using different artificial intelligence techniques since attackers are focusing on these areas. Second, this study presents the summarized literature and recent advancements in phishing attacks, which can be utilized to develop additional informational materials and advanced training programs for users to create awareness. Third, future studies should emphasize developing a hybrid phishing detection model and include the benefits of existing phishing detection methods. Fourth, future research should explore the applications of large language models (LLMs) in recognizing phishing attacks using the generated data.

6 Conclusion

Phishing is a cyber-attack that targets Internet users through social engineering and deception. Despite the availability of existing detection methods, phishing attacks continue to increase rapidly, causing millions of dollars in user losses and highlighting the need for more advanced detection techniques. To address this challenge, a web phishing dataset has been systematically created to explore additional unseen features and extend the existing studies. The study proposed a deep-leaning-based EGSO-CNN model with optimization techniques to enhance the performance with low false positive and error rates in phishing detection. The proposed model has been evaluated using three experimentations, yielded an accuracy of 99.44%, and is further validated with a different dataset. The model demonstrates strong potential for real-world applications, and its performance could be enhanced by exploring alternative deep learning approaches and incorporating transfer learning techniques with optimization. Ultimately, the proposed model can significantly aid organizations in implementing effective web phishing prevention strategies, thereby enhancing customer satisfaction and their reputation in an increasingly competitive marketplace.

Author contributions KB and SM conceptualize the topic. KB, SM, and RM are involved in Methodology, investigation, and validation. SM supervised the whole work. All authors reviewed the manuscript.

Funding Open access funding provided by Institute for Energy Technology.

Data and Code availability The process data and codes generated during and/or analyzed during the current study will be available at reasonable request.

Declarations

Conflict of interest The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Maware, C., Parsley, D.M., Huang, K., Swan, G.M., Akafuah, N.: Moving lab-based in-person training to online delivery: the case of a continuing engineering education program. J. Comput. Assist. Learn. **39**(4), 1167–1183 (2023). https://doi.org/10.1111/jcal.12789
2. Barik, K., Misra, S., Fernandez-Sanz, L.: A model for estimating resiliency of AI-based classifiers defending against cyber attacks. Int. J. Comput. Intell. Syst. **17**(1), 290 (2024). https://doi.org/10.1007/s44196-024-00686-3
3. James, J.W.: Engineering the Human Mind: Social Engineering Attack Using Kali Linux. SN Comput. Sci. **4**(6), 846 (2023). https://doi.org/10.1007/s42979-023-02321-y
4. Rahman, A.U., Al-Obeidat, F., Tubaishat, A., Shah, B., Anwar, S., Halim, Z.: Discovering the correlation between phishing susceptibility causing data biases and big five personality traits using C-GAN," IEEE Trans. Comput. Soc. Syst. (2022)
5. Chen, L., Peng, J., Liu, Y., Li, J., Xie, F., Zheng, Z.: Phishing scams detection in ethereum transaction network. ACM Trans. Internet Technol. TOIT **21**(1), 1–16 (2020)
6. Desolda, G., Ferro, L.S., Marrella, A., Catarci, T., Costabile, M.F.: Human factors in phishing attacks: a systematic literature review. ACM Comput. Surv. CSUR **54**(8), 1–35 (2021)
7. Barik, K., Misra, S.: IDS-Anta: an open-source code with a defense mechanism to detect adversarial attacks for intrusion detection system. Softw. Impacts **21**, 100664 (2024). https://doi.org/10.1016/j.simpa.2024.100664
8. Anti-Phishing Working Group, "Anti-Phishing Working Group,Q3,2024." 2024. [Online]. Available: https://docs.apwg.org/reports/apwg_trends_report_q3_2024.pdf
9. Gupta, B.B., Yadav, K., Razzak, I., Psannis, K., Castiglione, A., Chang, X.: A novel approach for phishing URLs detection using lexical based machine learning in a real-time environment. Comput. Commun. **175**, 47–57 (2021)
10. Sahingoz, O.K., Buber, E., Kugu, E.: DEPHIDES: deep learning based phishing detection system. IEEE Access **12**, 8052–8070 (2024). https://doi.org/10.1109/ACCESS.2024.3352629
11. Mumuni, A., Mumuni, F.: Automated data processing and feature engineering for deep learning and big data applications: a survey. J. Inf. Intell. (2024). https://doi.org/10.1016/j.jiixd.2024.01.002
12. Barik, K., Misra, S., Konar, K., Fernandez-Sanz, L., Koyuncu, M.: Cybersecurity deep: approaches, attacks dataset, and comparative study. Appl. Artif. Intell. **36**(1), 1 (2022). https://doi.org/10.1080/08839514.2022.2055399
13. Skula, I., Kvet, M.: A framework for preparing a balanced and comprehensive phishing dataset. IEEE Access **12**, 53610–53622 (2024). https://doi.org/10.1109/ACCESS.2024.3387437
14. Barik, K., Misra, S., Fernandez-Sanz, L.: Adversarial attack detection framework based on optimized weighted conditional stepwise adversarial network. Int. J. Inf. Secur. **23**(3), 2353–2376 (2024). https://doi.org/10.1007/s10207-024-00844-w
15. Naqvi, B., Perova, K., Farooq, A., Makhdoom, I., Oyedeji, S., Porras, J.: Mitigation strategies against the phishing attacks: a systematic literature review. Comput. Secur. 103387, (2023)
16. Ollmann, G.: The phishing guide understanding and preventing phishing attacks. NGS Softw. Insight Secur. Res. (2004)
17. Sunil, A.N.V., Sardana, A.: A pagerank based detection technique for phishing web sites. In: 2012 IEEE Symposium on Computers and Informatics (ISCI), IEEE, pp. 58–63 (2012)
18. Tan, C.C.L., Chiew, K.L., Yong, K.S.C., Sebastian, Y., Than, J.C.M., Tiong, W.K.: Hybrid phishing detection using joint visual and textual identity. Expert Syst. Appl. **220**, 119723 (2023)
19. Iqbal, F., Javed, A.R., Jhaveri, R.H., Almadhor, A., Farooq, U.: Transfer learning-based forensic analysis and classification of E-mail content. ACM Trans. Asian Low-Resour. Lang. Inf. Process. (2023). https://doi.org/10.1145/3604592
20. Biswas, B., Mukhopadhyay, A., Kumar, A., Delen, D.: A hybrid framework using explainable AI (XAI) in cyber-risk management for defence and recovery against phishing attacks. Decis. Support Syst. 114102 (2023)
21. Corona, I. et al.: DeltaPhish: detecting phishing webpages in compromised websites. In: Foley, S.N., Gollmann, D., Snekkenes, E. (eds.) Computer Security – ESORICS 2017, in Lecture Notes in Computer Science, vol. 10492, pp. 370–388, Springer International Publishing, Cham, (2017) https://doi.org/10.1007/978-3-319-66402-6_22.
22. Orunsolu, A.A., Sodiya, A.S., Akinwale, A.T.: A predictive model for phishing detection. J. King. Saud-Univ. Comput. Inf. Sci. **34**(2), 232–247 (2022)
23. Boyapati, M., Aygun, R.: Phishing Web Page Detection using Web Scraping. In: SoutheastCon 2023, IEEE, pp. 167–174 (2023)
24. Bacanin, N., et al.: Addressing feature selection and extreme learning machine tuning by diversity-oriented social network search: an application for phishing websites detection. Complex Intell. Syst. **9**(6), 7269–7304 (2023)
25. Uddin, M.M., Islam, K.A., Mamun, M., Tiwari, V.K., Park, J.: A comparative analysis of machine learning-based website phishing detection using URL information. In: 2022 5th International Conference on Pattern Recognition and Artificial Intelligence (PRAI), IEEE, pp. 220–224 (2022)
26. Marchal, S., Francois, J., State, R., Engel, T.: PhishStorm: detecting phishing with streaming analytics. IEEE Trans. Netw. Serv. Manag. **11**(4), 458–471 (2014). https://doi.org/10.1109/TNSM.2014.2377295
27. Ahammad, S.K.H., et al.: Phishing URL detection using machine learning methods. Adv. Eng. Softw. **173**, 103288 (2022)
28. Alani, M.M., Tawfik, H.: PhishNot: a cloud-based machine-learning approach to phishing URL detection. Comput. Netw. **218**, 109407 (2022)
29. Korkmaz, M., Sahingoz, O.K., Diri, B.: Detection of phishing websites by using machine learning-based URL analysis. In 2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT), IEEE, pp. 1–7 (2020)
30. Barraclough, P.A., Fehringer, G., Woodward, J.: Intelligent cyber-phishing detection for online. Comput. Secur. **104**, 102123 (2021)
31. Jain, A.K., Debnath, N., Jain, A.K.: APuML: an efficient approach to detect mobile phishing webpages using machine learning. Wirel. Pers. Commun. **125**(4), 3227–3248 (2022)
32. Sanchez-Paniagua, M., Fernandez, E.F., Alegre, E., Al-Nabki, W., Gonzalez-Castro, V.: Phishing URL detection: a real-case scenario through login URLs. IEEE Access **10**, 42949–42960 (2022). https://doi.org/10.1109/ACCESS.2022.3168681
33. Jha, A.K., Muthalagu, R., Pawar, P.M.: Intelligent phishing website detection using machine learning. Multimed. Tools Appl. **82**(19), 29431 (2023)
34. Karim, A., Shahroz, M., Mustofa, K., Belhaouari, S.B., Joga, S.R.K.: Phishing detection system through hybrid machine learning based on URL. IEEE Access **11**, 36805–36822 (2023)
35. Shirazi, H., Muramudalige, S.R., Ray, I., Jayasumana, A.P., Wang, H.: Adversarial autoencoder data synthesis for enhancing machine learning-based phishing detection algorithms. IEEE Trans. Serv. Comput. **16**(4), 2411–2422 (2023)
36. Pandey, P., Mishra, N.: Phish-Sight: a new approach for phishing detection using dominant colors on web pages and machine learning. Int. J. Inf. Secur. **22**(4), 881–891 (2023)
37. Smadi, S., Aslam, N., Zhang, L.: Detection of online phishing email using dynamic evolving neural network based on reinforcement learning. Decis. Support. Syst. **107**, 88–102 (2018)

38. Yang, P., Zhao, G., Zeng, P.: Phishing website detection based on multidimensional features driven by deep learning. IEEE Access **7**, 15196–15209 (2019)
39. Zhu, E., Ju, Y., Chen, Z., Liu, F., Fang, X.: DTOF-ANN: an artificial neural network phishing detection model based on decision tree and optimal features. Appl. Soft Comput. **95**, 106505 (2020)
40. Tang, L., Mahmoud, Q.H.: A deep learning-based framework for phishing website detection. IEEE Access **10**, 1509–1521 (2021)
41. Al-Ahmadi, S., Alotaibi, A., Alsaleh, O.: PDGAN: Phishing detection with generative adversarial networks. IEEE Access **10**, 42459–42468 (2022)
42. Almousa, M., Zhang, T., Sarrafzadeh, A., Anwar, M.: Phishing website detection: How effective are deep learning-based models and hyperparameter optimization? Secur. Priv. **5**(6), e256 (2022)
43. Hussain, M., Cheng, C., Xu, R., Afzal, M.: CNN-Fusion: An effective and lightweight phishing detection method based on multi-variant ConvNet. Inf. Sci. **631**, 328–345 (2023)
44. Prabakaran, M.K., Meenakshi Sundaram, P., Chandrasekar, A.D.: An enhanced deep learning-based phishing detection mechanism to effectively identify malicious URLs using variational autoencoders. IET Inf. Secur. **17**(3), 423–440 (2023)
45. Kumar, P.P., Jaya, T., Rajendran, V.: SI-BBA–A novel phishing website detection based on Swarm intelligence with deep learning. Mater. Today Proc. **80**, 3129–3139 (2023)
46. Opara, C., Chen, Y., Wei, B.: Look before You leap: Detecting phishing web pages by exploiting raw URL And HTML characteristics. Expert Syst. Appl. **236**, 121183 (2024)
47. Sarker, O., Jayatilaka, A., Haggag, S., Liu, C., Babar, M.A.: A Multi-vocal Literature Review on challenges and critical success factors of phishing education, training and awareness. J. Syst. Softw. **208**, 111899 (2024). https://doi.org/10.1016/j.jss.2023.111899
48. Sonowal, G.: Phishing and Communication Channels: A Guide to Identifying and Mitigating Phishing Attacks. Apress, Berkeley (2022). https://doi.org/10.1007/978-1-4842-7744-7
49. Tabassum, H., Ghosh, G., Atika, A., Chakrabarty, A.: Detecting online recruitment fraud using machine learning. In 2021 9th International Conference on Information and Communication Technology (ICoICT), IEEE, pp. 472–477 (2021)
50. Vidyasri, P., Suresh, S.: FDN-SA: fuzzy deep neural-stacked autoencoder-based phishing attack detection in social engineering. Comput. Secur. **148**, 104188 (2025). https://doi.org/10.1016/j.cose.2024.104188
51. Xi, P., Cheng, D., Lu, G., Deng, Z., Zhang, G., Zhang, S.: Identifying local useful information for attribute graph anomaly detection. Neurocomputing **617**, 128900 (2025). https://doi.org/10.1016/j.neucom.2024.128900
52. He, D., Lv, X., Zhu, S., Chan, S., Choo, K.-K.R.: A Method for Detecting Phishing Websites Based on Tiny-Bert Stacking. IEEE Internet Things J. **11**(2), 2236–2243 (2024). https://doi.org/10.1109/JIOT.2023.3292171
53. Booth, T.M., Ghosh, S.: A gradient descent multi-algorithm grid search optimization of deep learning for sensor fusion. In 2023 IEEE International Systems Conference (SysCon), IEEE, pp. 1–8 (2023)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.